Đại học Quốc gia Thành phố Hồ Chí Minh
Trường Đại học Bách khoa
Khoa Khoa học và Kỹ thuật Máy tính



Học sâu và ứng dụng trong thị giác máy tính (CO5085)

Bài tập lớn A1 - Báo cáo cuối (A1.1 + A1.2)

CNN hay Transformer? So sánh ResNet-18 và DeiT-Tiny
trên CIFAR-100 với nhiều chế độ huấn luyện

Giảng viên: Thầy Lê Thành Sách
Thầy Nguyễn Quốc Minh

Thành viên nhóm: Đàm Quang Phục - 2570483
Hồ Bảo An - 2570164

Mã nguồn:
`https://github.com/dqpinfo/Deep_Learning_Computer_Vision_CO5085/tree/main/Assignment_1`

Thành Phố Hồ Chí Minh, 11/2026

Mục lục

1 Chủ đề và mục tiêu **3**

2 Tập dữ liệu: CIFAR-100 **3**
2.1 Lý do chọn . . . 3
2.2 Nguồn và license . . . 3
2.3 Cách chia split . . . 4
2.4 Khám phá dữ liệu (EDA) . . . 4

3 DataLoader và augmentation **6**

4 Mô hình sử dụng **7**

5 Protocol thí nghiệm **8**
5.1 Ma trận 8 thí nghiệm . . . 8
5.2 Huấn luyện . . . 9
5.3 Metric và câu hỏi nghiên cứu . . . 9

6 Kiểm tra pipeline ở mốc A1.1 **9**

7 Môi trường chạy A1.2 **10**

8 Kết quả **10**
8.1 Bảng tổng hợp . . . 10
8.2 Quá trình học . . . 12
8.3 Chất lượng so với chi phí . . . 13

9 Phân tích lỗi **13**
9.1 Nhầm lẫn theo siêu lớp . . . 14
9.2 Các cặp lớp hay nhầm nhất . . . 16
9.3 Hai mô hình có sai cùng chỗ không? . . . 16

10 Thảo luận **18**
10.1 ViT có cần pretrain không? . . . 18
10.2 Khi nào đóng băng có lợi? . . . 18
10.3 Chi phí so với chất lượng . . . 18
10.4 Hạn chế . . . 18

11 Tái lập kết quả **19**

12 Sử dụng công cụ AI **19**

Mục lục

Tóm tắt. Nhóm so sánh **ResNet-18** (CNN) và **DeiT-Tiny** (Transformer) trên **CIFAR-100** (45.000 / 5.000 / 10.000 ảnh train / validation / test, split cố định seed 42) qua **8 thí nghiệm**: 2 kiến trúc × 4 chế độ (học từ đầu, chỉ train head, đóng băng một phần, fine-tune toàn bộ). Mục 1–6 là phần A1.1 (dataset và protocol, hạn 26/10/2026); Mục 7–11 là phần A1.2 (kết quả và thảo luận, hạn 09/11/2026). Kết quả chính: cấu hình tốt nhất là `deit_tiny_full` (test accuracy 85,02%, 11 phút trên T4), tiếp theo là `deit_tiny_partial` (81,31%) và `resnet18_full` (74,71%). Pretrain quan trọng với ViT hơn hẳn với CNN: học từ đầu, DeiT-Tiny chỉ đạt 48,48% (kém ResNet-18 67,46%), nhưng khi dùng trọng số pretrained, DeiT-Tiny vượt ResNet-18 ở cả ba mức đóng băng. Đóng băng một phần gần bằng fine-tune toàn bộ với ResNet-18 (kém 1,0 điểm, nhanh gấp đôi) nhưng kém 3,7 điểm với DeiT-Tiny; chỉ train head kém xa ở cả hai họ. Mọi con số A1.2 được `analyze.py` sinh tự động từ log huấn luyện (`output/runs/*.json`).

1. Chủ đề và mục tiêu

Bài A1 so sánh hai họ kiến trúc cho bài toán phân loại ảnh: mạng tích chập (CNN) và Vision Transformer (ViT) [3]. Hai họ được đánh giá dưới nhiều chế độ huấn luyện: học từ đầu (*from-scratch*) so với dùng trọng số *pretrained*, và ba mức đóng băng backbone trên bản pretrained (chỉ train head, đóng băng một phần, fine-tune toàn bộ).

Báo cáo gồm hai phần. Phần A1.1 (Mục 1–6) chốt tập dữ liệu, cách chia split, DataLoader và augmentation, các mô hình sử dụng và protocol thí nghiệm. Phần A1.2 (Mục 7–11) trình bày môi trường chạy, kết quả 8 thí nghiệm, phân tích lỗi và thảo luận. Protocol ở phần A1.1 được giữ nguyên khi chạy A1.2; các thay đổi nhỏ về cách cài đặt (không đổi protocol) được ghi ở Mục 7.

2. Tập dữ liệu: CIFAR-100

2.1. Lý do chọn

- **Đúng ràng buộc của đề:** tập ảnh công khai, có 100 lớp (yêu cầu ≥ 10) và 50.000 ảnh huấn luyện; không trùng với các tập đã dùng ở E1 (MNIST / Fashion-MNIST / CIFAR-10).
- **Đủ khó để so sánh có ý nghĩa:** mỗi lớp chỉ có 500 ảnh train, ảnh nhỏ 32×32 và nhiều lớp rất giống nhau (ví dụ 5 loại cây: maple, oak, palm, pine, willow). Trong điều kiện này, ưu thế của pretrained và nhu cầu dữ liệu của Transformer bộc lộ rõ.
- **Vừa sức tính toán:** toàn bộ tập khoảng 140 MB; cả 8 cấu hình có thể chạy trên một GPU phổ thông (Google Colab T4) trong thời gian của mốc A1.2.
- **Có nhãn hai cấp:** 100 lớp chi tiết được gom thành 20 siêu lớp, giúp phân tích lỗi kiểu "nhầm trong cùng nhóm" hay "nhầm khác nhóm".

2.2. Nguồn và license

CIFAR-100 do Krizhevsky (2009) công bố tại `https://www.cs.toronto.edu/~kriz/cifar.html`, là tập con có gán nhãn của bộ "80 Million Tiny Images". Tập không đi kèm license chuẩn (kiểu MIT hay CC); tác giả cho phép dùng tự do cho nghiên cứu và học tập, với yêu cầu trích dẫn báo cáo kỹ thuật gốc [1]. Bộ 80 Million Tiny Images gốc đã bị gỡ năm 2020 do chứa nhãn không phù hợp, nhưng CIFAR-10/100 là tập đã được gán nhãn lại thủ công và vẫn được cung cấp chính thức.

Do server gốc tải rất chậm từ Việt Nam, nhóm tải dữ liệu từ bản sao chính thức trên Hugging Face (`uoft-cs/cifar100`). Bản này có cùng ảnh, cùng nhãn và cùng thứ tự với

bản gốc. Ảnh được giải mã một lần và cache vào `data/cifar100.npz` (`data.py`); thư mục `data/` không được commit lên GitHub. Metadata của bản Hugging Face ghi sai tên lớp `crab` thành `cra`; `data.py` sửa lại tên gốc khi nạp.

2.3. Cách chia split

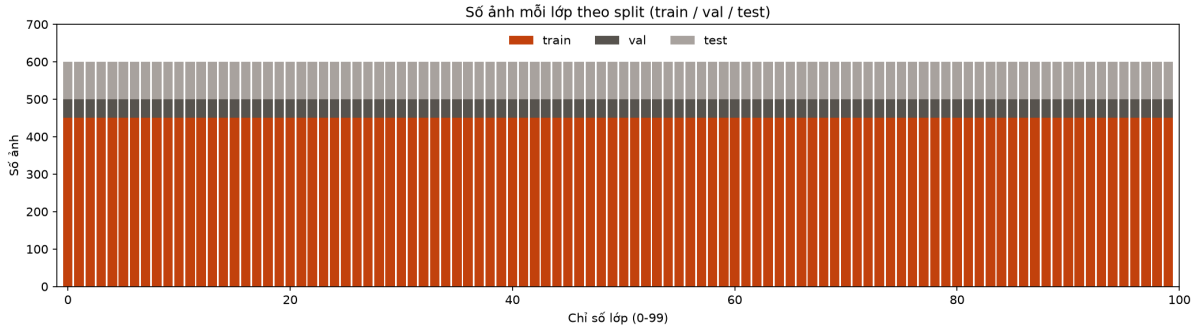
Bảng 1: Ba tập dữ liệu dùng chung cho cả 8 thí nghiệm (`output/eda/summary.json`).

Split	Số ảnh	Mỗi lớp	Nguồn	Vai trò
Train	45.000	450	Tập train gốc	Cập nhật trọng số
Validation	5.000	50	Tập train gốc	Chọn checkpoint tốt nhất, theo dõi quá trình học
Test	10.000	100	Tập test gốc	Chỉ dùng một lần ở cuối mỗi thí nghiệm

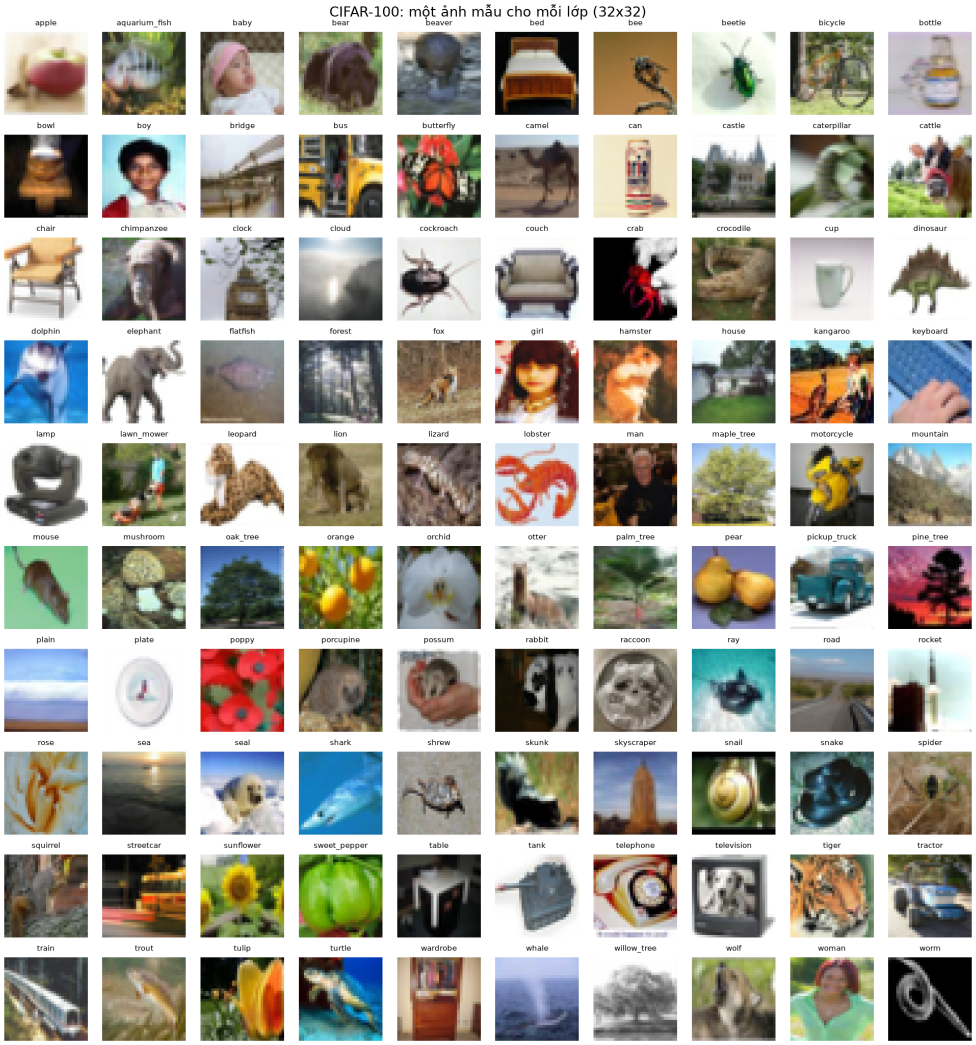
Validation được lấy **phân tầng**: mỗi lớp chọn ngẫu nhiên 50 ảnh từ tập train gốc với seed 42 (`make_split` trong `data.py`). Chỉ số ảnh được lưu cố định trong `splits/cifar100_split.json`, nên cả 8 thí nghiệm dùng **đúng cùng một split**. Tập test không được dùng để chọn siêu tham số hay checkpoint.

2.4. Khám phá dữ liệu (EDA)

Script `eda.py` đếm số ảnh mỗi lớp theo split, vẽ ảnh mẫu, ví dụ augmentation, tính thống kê pixel và liệt kê 20 siêu lớp; kết quả lưu ở `output/eda/`.



Hình 1: Số ảnh mỗi lớp theo split: dữ liệu cân bằng tuyệt đối (450 / 50 / 100 ảnh mỗi lớp).



Hình 2: Một ảnh mẫu cho mỗi lớp của CIFAR-100 (độ phân giải gốc 32×32).

Nhận xét.

- Mọi ảnh là ảnh màu RGB kích thước $32 \times 32 \times 3$. Không lớp nào thiếu ảnh (Hình 1), nên accuracy và macro-F1 phản ánh tương đương nhau; nhóm vẫn báo cáo cả hai.
- Trung bình pixel trên tập train (R, G, B) là (0.507, 0.487, 0.441), độ lệch chuẩn là (0.267, 0.256, 0.276). Thống kê này gần với thống kê ImageNet (mean (0.485, 0.456, 0.406), std (0.229, 0.224, 0.225)), nên việc dùng chuẩn hoá ImageNet cho mọi cấu hình là hợp lý (Mục 3).
- Ảnh rất nhỏ, vật thể thường chiếm gần hết khung hình và nhiều chi tiết bị mất. Lỗi dự kiến tập trung ở các lớp cùng siêu lớp (Bảng 2), ví dụ các loài cây, các loại xe hay các loài thú có vú cỡ nhỏ.

Bảng 2: 20 siêu lớp của CIFAR-100, mỗi siêu lớp gồm 5 lớp chi tiết.

Siêu lớp	Các lớp chi tiết
aquatic mammals	beaver, dolphin, otter, seal, whale
fish	aquarium fish, flatfish, ray, shark, trout
flowers	orchid, poppy, rose, sunflower, tulip

Siêu lớp	Các lớp chi tiết
food containers	bottle, bowl, can, cup, plate
fruit and vegetables	apple, mushroom, orange, pear, sweet pepper
household electrical devices	clock, keyboard, lamp, telephone, television
household furniture	bed, chair, couch, table, wardrobe
insects	bee, beetle, butterfly, caterpillar, cockroach
large carnivores	bear, leopard, lion, tiger, wolf
large man-made outdoor things	bridge, castle, house, road, skyscraper
large natural outdoor scenes	cloud, forest, mountain, plain, sea
large omnivores and herbivores	camel, cattle, chimpanzee, elephant, kangaroo
medium mammals	fox, porcupine, possum, raccoon, skunk
non-insect invertebrates	crab, lobster, snail, spider, worm
people	baby, boy, girl, man, woman
reptiles	crocodile, dinosaur, lizard, snake, turtle
small mammals	hamster, mouse, rabbit, shrew, squirrel
trees	maple tree, oak tree, palm tree, pine tree, willow tree
vehicles 1	bicycle, bus, motorcycle, pickup truck, train
vehicles 2	lawn mower, rocket, streetcar, tank, tractor

3. DataLoader và augmentation

Bảng 3: Pipeline tiền xử lý, giống hệt nhau cho cả 8 cấu hình (`data.py`).

Bước	Train	Validation / Test
1. Augmentation ở 32×32	`RandomCrop(32, padding=4)` + `RandomHorizontalFlip`	Không
2. Phóng ảnh	Nội suy bicubic lên 224×224 (chạy trên GPU, xem Mục 7)	
3. Chuẩn hoá	Chuẩn hoá theo mean/std của ImageNet	
DataLoader	batch 128, `shuffle`, `drop_last`	batch 128, không shuffle



Hình 3: Một ảnh gốc và các phiên bản sau augmentation (đã phóng lên 224×224).

Lý do thiết kế.

- **Phóng lên** 224×224**:** trọng số pretrained của cả hai mô hình được học ở 224×224. Riêng DeiT-Tiny dùng patch 16×16; nếu giữ ảnh 32×32 thì chỉ còn 4 token, không đủ để attention hoạt động có ý nghĩa.
- **Cùng kích thước, cùng augmentation, cùng chuẩn hoá cho cả 8 cấu hình** (kể cả from-scratch): khác biệt kết quả chỉ đến từ ba yếu tố cần so sánh là kiến trúc, cách khởi tạo và mức đóng băng.
- **Augmentation nhẹ:** crop + flip là chuẩn phổ biến cho CIFAR. Nhóm cố ý không dùng augmentation mạnh (RandAugment, Mixup, ...) để không ưu ái riêng Transformer from-scratch, vốn hưởng lợi nhiều từ augmentation mạnh. Đây là hạn chế có chủ đích và sẽ được thảo luận ở A1.2.

4. Mô hình sử dụng

Bảng 4: Hai mô hình đại diện cho hai họ kiến trúc (`config.py`, `models.py`).

Họ	Mô hình (tên trong `timm`)	Tham số (head 100 lớp)	Lý do chọn
CNN	ResNet-18 [2] `resnet18.a1_in1k`	11.227.812	CNN chuẩn, nhỏ, có kết nối tắt (residual); baseline phổ biến nhất
Transformer	DeiT-Tiny/16 [4] `deit_tiny_patch16_224.fb_in1k`	5.543.716	Một trong các ViT nhỏ nhất có trọng số ImageNet-1k tốt, cùng cỡ với ResNet-18, chạy được trên GPU phổ thông

Hai mô hình có cùng nguồn pretrained (chỉ ImageNet-1k, không dùng ImageNet-21k), lấy qua thư viện `timm` [5], và cùng bậc kích thước, để so sánh không bị lệch vì một bên được học trước trên nhiều dữ liệu hơn. Lớp phân loại cuối của cả hai được thay bằng lớp mới 100 đầu ra, khởi tạo ngẫu nhiên.

5. Protocol thí nghiệm

5.1. Ma trận 8 thí nghiệm

Bảng 5: 8 cấu hình và số tham số trainable, đo bằng `smoke_test.py` (`output/protocol_table.json`).

Cấu hình	Kiến trúc	Khởi tạo	Chế độ	Trainable	Tỉ lệ
`resnet18_scratch`	ResNet-18	Ngẫu nhiên	Học từ đầu	11.227.812	100,0%
`resnet18_head`	ResNet-18	ImageNet-1k	Chỉ train head	51.300	0,5%
`resnet18_partial`	ResNet-18	ImageNet-1k	Đóng băng một phần	10.544.740	93,9%
`resnet18_full`	ResNet-18	ImageNet-1k	Fine-tune toàn bộ	11.227.812	100,0%
`deit_tiny_scratch`	DeiT-Tiny	Ngẫu nhiên	Học từ đầu	5.543.716	100,0%
`deit_tiny_head`	DeiT-Tiny	ImageNet-1k	Chỉ train head	19.300	0,4%
`deit_tiny_partial`	DeiT-Tiny	ImageNet-1k	Đóng băng một phần	2.688.868	48,5%
`deit_tiny_full`	DeiT-Tiny	ImageNet-1k	Fine-tune toàn bộ	5.543.716	100,0%

Định nghĩa các mức đóng băng (áp dụng trên bản pretrained, `models.py`):

- **Chỉ train head** (linear probe): đóng băng toàn bộ backbone, chỉ học lớp phân loại cuối.
- **Đóng băng một phần:** khoảng nửa đầu của mạng. ResNet-18 đóng băng `conv1`, `bn1`, `layer1`, `layer2` và train `layer3`, `layer4`, `fc`. DeiT-Tiny đóng băng `patch_embed`, positional embedding, token CLS và 6/12 block đầu; train 6 block cuối, LayerNorm cuối và head.
- **Fine-tune toàn bộ:** mọi tham số đều được cập nhật.

Hai mô hình được đóng băng cùng "một nửa độ sâu" (2/4 stage của ResNet-18, 6/12 block của DeiT-Tiny), nhưng tỉ lệ tham số trainable khác nhau: 93,9% so với 48,5%. Lý do là ResNet tập trung phần lớn tham số ở các stage sâu (`layer3`, `layer4`), còn các block của DeiT có kích thước bằng nhau. Nhóm sẽ báo cáo cả hai góc nhìn (độ sâu và số tham số) khi thảo luận ở A1.2. Với ResNet-18, các lớp BatchNorm thuộc phần bị đóng băng được giữ ở chế độ `eval` trong lúc train (`set_train_mode`), để thống kê pretrained không bị thay đổi.

5.2. Huấn luyện

Bảng 6: Siêu tham số huấn luyện, dùng đúng như kế hoạch A1.1 (`config.TRAIN_HPARAMS`).

Thiết lập	From-scratch	Pretrained (3 chế độ)
Optimizer	AdamW, weight decay 0.05	
Learning rate	10^{-3}	10^{-4}
Lịch learning rate	Warmup tuyến tính 1 epoch, sau đó cosine giảm về 0	
Số epoch	30	10
Loss	Cross-entropy, label smoothing 0.1	
Batch size / seed	128 / 42	
Chọn checkpoint	Epoch có validation accuracy cao nhất; sau đó đánh giá trên test đúng một lần	

Vòng lặp huấn luyện được tự viết bằng PyTorch trong `train.py`: xoá gradient → forward → tính loss → backward → `optimizer.step()`. Thiết bị được chọn tự động theo thứ tự CUDA, MPS, CPU. From-scratch được học nhiều epoch hơn và với learning rate lớn hơn, vì mô hình phải học mọi thứ từ đầu. Để so sánh công bằng trong từng nhóm, các cấu hình cùng nhóm dùng chung siêu tham số.

5.3. Metric và câu hỏi nghiên cứu

- **Chất lượng:** top-1 accuracy và macro-F1 (`sklearn.metrics.f1_score`) trên validation và test; đường cong loss/accuracy theo epoch (`train.py` đã ghi đủ các giá trị này vào log JSON của mỗi lần chạy).
- **Chi phí:** thời gian huấn luyện, số tham số trainable, bộ nhớ GPU đỉnh (`torch.cuda.max_memory_allocated`).
- **Phân tích lỗi:** confusion matrix ở mức 20 siêu lớp (dễ đọc hơn ma trận 100×100), các cặp lớp hay nhầm nhất và ảnh mẫu bị đoán sai của cấu hình tốt nhất mỗi họ. Dự đoán trên test được lưu sẵn (`*_test_preds.json`) để phục vụ phân tích này.

Câu hỏi cần trả lời và giả thuyết ban đầu (đặt ra ở A1.1, trước khi có số liệu; được kiểm chứng ở Mục 10):

1. **ViT có cần pretrain không?** Dự đoán DeiT-Tiny from-scratch kém hơn rõ rệt so với ResNet-18 from-scratch, vì Transformer thiếu "thiên kiến cục bộ" của tích chập và CIFAR-100 chỉ có 45.000 ảnh train.
2. **Khi nào đóng băng có lợi?** Dự đoán đóng băng một phần đạt gần bằng fine-tune toàn bộ với ít tham số trainable hơn. Chỉ train head có thể kém hơn, vì ảnh 32×32 phóng lên khác khá xa ảnh ImageNet.
3. **Chi phí so với chất lượng:** so sánh accuracy trên mỗi đơn vị thời gian huấn luyện giữa các cấu hình.

6. Kiểm tra pipeline ở mốc A1.1

Script `smoke_test.py` tạo đủ 8 cấu hình, đếm tham số và chạy một bước forward + backward trên một batch 8 ảnh ngẫu nhiên 224×224. Script xác nhận các tham số bị đóng băng không nhận gradient, còn các tham số trainable thì có gradient. Cả 8 cấu hình đều đạt (cột "Kiểm tra" = OK trong `output/protocol_table.md`; số tham số ở Bảng 5).

Ngoài ra, nhóm chạy thử toàn bộ vòng lặp huấn luyện – đánh giá trên tập con nhỏ (256 ảnh mỗi tập, 1 epoch, thiết bị `mps`) để chắc chắn pipeline chạy thông: `deit_tiny_partial` (khoảng 6 giây) và `resnet18_head` (khoảng 3 giây). Huấn luyện, đánh giá validation, chọn checkpoint và đánh giá test đều chạy không lỗi; log nằm ở `output/smoke/`. Accuracy của các lần chạy này chỉ ở mức đoán ngẫu nhiên (khoảng 1–3%) vì chỉ có 1 epoch trên 256 ảnh: đây là kiểm tra code, **không phải kết quả thí nghiệm**.

7. Môi trường chạy A1.2

Cả 8 thí nghiệm được chạy trên Google Colab với GPU Tesla T4, bằng notebook `colab_a1_2.ipynb` (gọi `run_all.py`, lần lượt chạy `train.py` cho từng cấu hình). Thứ tự chạy: 6 cấu hình pretrained trước, 2 cấu hình from-scratch sau. Máy Mac M1 8 GB của nhóm không đủ cho protocol 224×224: đo thử, một epoch ResNet-18 from-scratch mất khoảng 17 phút, còn DeiT-Tiny from-scratch vượt quá bộ nhớ và chậm hơn khoảng 100 lần.

So với mô tả ở A1.1, nhóm **giữ nguyên protocol** (split, augmentation, kích thước ảnh, mô hình, mức đóng băng, siêu tham số ở Bảng 6) và chỉ thay đổi ba chi tiết cài đặt để chạy được trên Colab:

- **Phóng ảnh trên GPU.** DataLoader chỉ làm crop + lật ở 32×32; bước phóng bicubic lên 224×224 và chuẩn hoá (`to_model_input` trong `data.py`) chạy trên GPU, vì 2 lõi CPU của Colab không phóng ảnh kịp cho GPU. Kết quả lệch so với cách phóng bằng PIL trung bình khoảng 1 mức xám trên 255 (PIL làm tròn về số nguyên 8 bit và dùng hệ số bicubic hơi khác), nên không ảnh hưởng tới so sánh.
- **Mixed precision (AMP, fp16)** [6] cho forward/backward, kèm `GradScaler` để tránh gradient quá nhỏ bị làm tròn về 0. Áp dụng như nhau cho cả 8 cấu hình.
- **Checkpoint sau mỗi epoch.** Phiên Colab miễn phí có thể bị ngắt; chạy lại cùng lệnh sẽ tiếp tục từ epoch cuối (trạng thái mô hình, optimizer, lịch learning rate, bộ sinh số ngẫu nhiên đều được lưu). Nhóm đã kiểm tra: một lần chạy bị dừng giữa chừng rồi chạy tiếp cho kết quả trùng khớp từng epoch với lần chạy liền một mạch.

Cách đo chi phí. Thời gian huấn luyện là tổng thời gian các epoch train (không tính đánh giá validation), đo với `torch.cuda.synchronize()`; bộ nhớ là `torch.cuda.max_memory_allocated()`; tốc độ dự đoán là số ảnh test xử lý mỗi giây ở batch 128 (đã gồm bước phóng ảnh, bỏ batch đầu tiên để GPU "khởi động").

8. Kết quả

8.1. Bảng tổng hợp

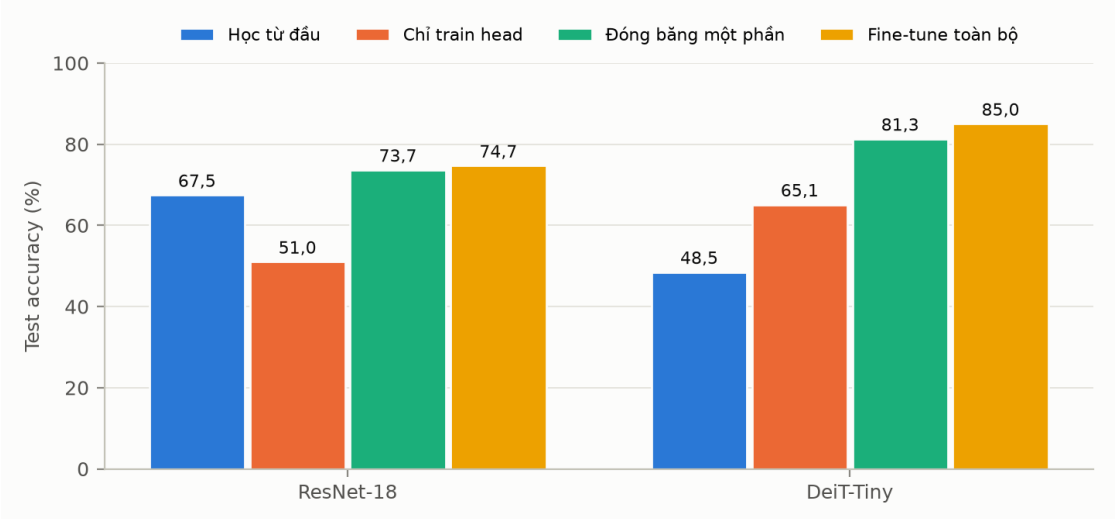
Mỗi cấu hình được đánh giá trên test đúng một lần, bằng checkpoint có validation accuracy cao nhất.

Bảng 7: Chất lượng của 8 cấu hình (%). "Epoch tốt nhất": epoch có validation accuracy cao nhất / tổng số epoch; "Train acc": accuracy trên tập train ở epoch cuối (có augmentation).

Cấu hình	Chế độ	Epoch tốt nhất	Train acc	Val acc	Test acc	Test macro-F1
`resnet18_scratch`	Học từ đầu	27/30	98,59	68,54	67,46	67,50
`resnet18_head`	Chỉ train head	10/10	51,17	50,82	51,02	49,24
`resnet18_partial`	Đóng băng một phần	9/10	76,03	74,42	73,67	73,42
`resnet18_full`	Fine-tune toàn bộ	9/10	77,04	74,92	74,71	74,48
`deit_tiny_scratch`	Học từ đầu	20/30	96,49	47,86	48,48	48,01
`deit_tiny_head`	Chỉ train head	10/10	65,16	65,62	65,11	64,72
`deit_tiny_partial`	Đóng băng một phần	7/10	93,40	80,76	81,31	81,30
`deit_tiny_full`	Fine-tune toàn bộ	10/10	97,18	84,94	85,02	85,04

Bảng 8: Chi phí của 8 cấu hình trên GPU Tesla T4.

Cấu hình	Tham số trainable	Train (phút)	Giây / epoch	Bộ nhớ GPU (GB)	Dự đoán (ảnh/s)
`resnet18_scratch`	11.227.812 (100,0%)	37,0	74	1,69	1.873
`resnet18_head`	51.300 (0,5%)	3,9	23	0,60	1.908
`resnet18_partial`	10.544.740 (93,9%)	6,0	36	0,72	1.898
`resnet18_full`	11.227.812 (100,0%)	12,3	74	1,69	1.879
`deit_tiny_scratch`	5.543.716 (100,0%)	32,9	66	2,23	1.964
`deit_tiny_head`	19.300 (0,3%)	3,9	23	0,25	1.969
`deit_tiny_partial`	2.688.868 (48,5%)	7,2	43	1,19	1.956
`deit_tiny_full`	5.543.716 (100,0%)	11,0	66	2,23	1.958

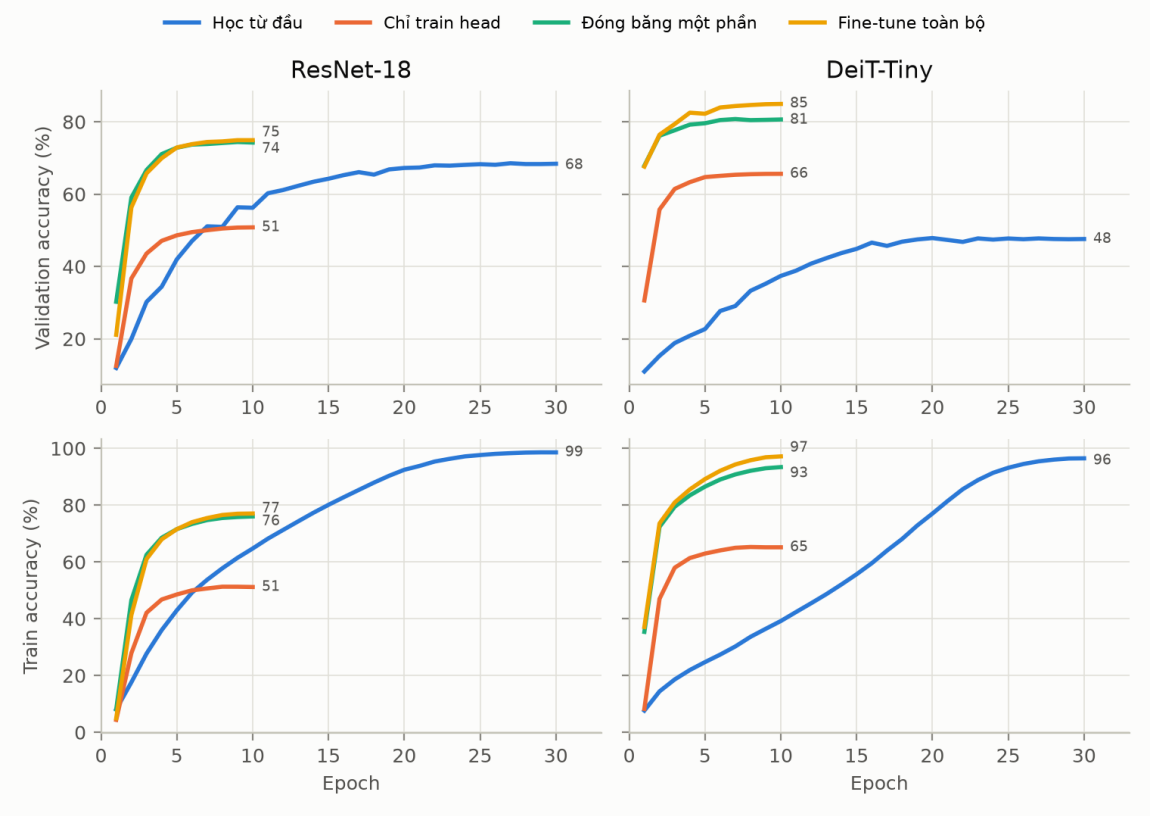


Hình 4: Test accuracy theo kiến trúc và chế độ huấn luyện.

Nhận xét.

- **Thứ hạng trong mỗi họ.** Với cả hai họ, fine-tune toàn bộ đứng đầu, đóng băng một phần đứng thứ hai. Hai họ khác nhau ở hai chế độ còn lại: với ResNet-18, học từ đầu (67,46%) tốt hơn hẳn chỉ train head (51,02%); với DeiT-Tiny thì ngược lại, chỉ train head (65,11%) tốt hơn học từ đầu (48,48%).
- **Khoảng cách giữa hai họ.** Khi học từ đầu, ResNet-18 hơn DeiT-Tiny 19,0 điểm. Khi dùng trọng số pretrained, DeiT-Tiny hơn ResNet-18 ở mọi mức đóng băng: +14,1 điểm (chỉ head), +7,6 (một phần), +10,3 (toàn bộ), dù DeiT-Tiny chỉ có khoảng một nửa số tham số (5,5 triệu so với 11,2 triệu).
- **Macro-F1 bám sát accuracy** (chênh lệch tối đa 1,8 điểm, ở `resnet18_head`) vì test có đúng 100 ảnh mỗi lớp. Test accuracy cũng lệch validation accuracy không quá 1,1 điểm ở mọi cấu hình, nên việc chọn checkpoint theo validation không làm kết quả test bị lạc quan.

8.2. Quá trình học



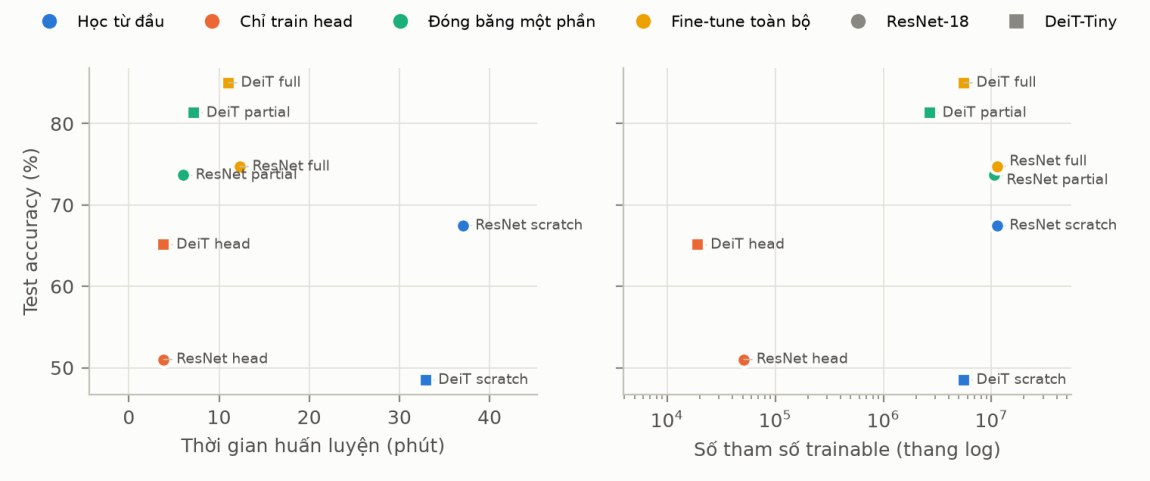
Hình 5: Validation accuracy (hàng trên) và train accuracy (hàng dưới) theo epoch; số ở cuối mỗi đường là giá trị epoch cuối.

Nhận xét.

- **Tốc độ hội tụ.** Các cấu hình pretrained đạt gần mức cuối chỉ sau 3–5 epoch (DeiT-Tiny đóng băng một phần hoặc toàn bộ đã có khoảng 76% validation accuracy sau epoch 2). Hai cấu hình học từ đầu cần khoảng 20 epoch mới chững lại; epoch tốt nhất là 27 (ResNet-18) và 20 (DeiT-Tiny).
- **Overfitting khi học từ đầu.** Train accuracy ở epoch cuối gần 100% (ResNet-18 98,59%, DeiT-Tiny 96,49%) trong khi validation dừng ở khoảng 68% và 48%. Với DeiT-Tiny, validation gần như đứng yên từ epoch 16 trong khi train vẫn tăng thêm khoảng 40 điểm: mô hình học thuộc tập train thay vì học đặc trưng tổng quát.

- **Pretrained ít overfit hơn.** ResNet-18 đóng băng một phần hoặc toàn bộ có train accuracy chỉ cao hơn validation khoảng 2 điểm. Train accuracy của chúng chỉ khoảng 77%, tức mô hình chưa khớp hết tập train: ResNet-18 pretrained có thể còn tăng nếu train lâu hơn hoặc dùng learning rate lớn hơn 10^{-4}. DeiT-Tiny fine-tune toàn bộ có khoảng cách lớn hơn (97,18% so với 84,94%) nhưng validation vẫn tăng nhẹ đến epoch cuối (epoch tốt nhất là 10).
- **Chỉ train head bị chặn trên.** Train và validation accuracy gần bằng nhau và đi ngang sau khoảng epoch 6: một lớp tuyến tính trên đặc trưng cố định không đủ biểu diễn 100 lớp CIFAR-100 (underfitting chứ không phải overfitting).

8.3. Chất lượng so với chi phí



Hình 6: Test accuracy theo thời gian huấn luyện (trái) và theo số tham số trainable (phải, thang log). Màu là chế độ huấn luyện, hình tròn là ResNet-18, hình vuông là DeiT-Tiny.

Nhận xét.

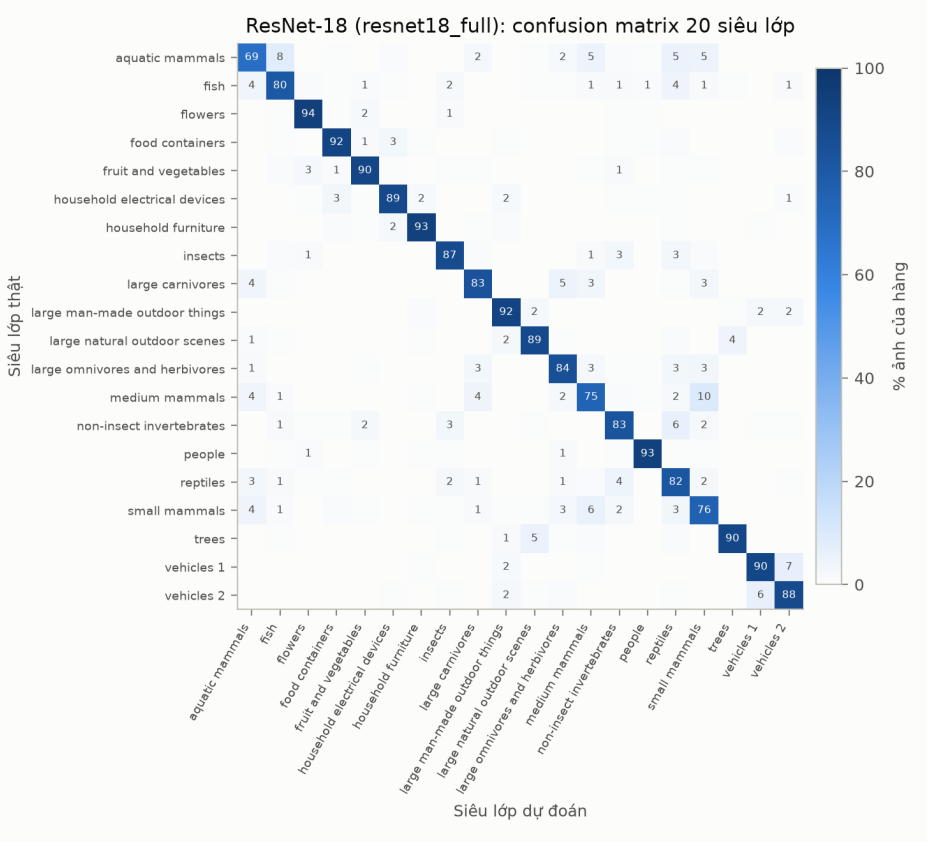
- **Thời gian.** Hai cấu hình học từ đầu tốn nhiều thời gian nhất (33–37 phút, 30 epoch) nhưng lại thuộc nhóm kém nhất. `deit_tiny_full` vừa chính xác nhất vừa nhanh hơn `resnet18_full` (11,0 so với 12,3 phút). `deit_tiny_partial` đạt 81,31% chỉ trong 7,2 phút, là lựa chọn tốt nếu thời gian bị giới hạn.
- **Số tham số trainable không phản ánh hết chi phí.** `resnet18_partial` vẫn train 93,9% tham số nhưng nhanh gấp đôi `resnet18_full` và tốn chưa đến một nửa bộ nhớ, vì các tầng đầu bị đóng băng là các tầng có feature map lớn nhất (từ 112×112 xuống 28×28): bỏ được lan truyền ngược qua chúng mới là phần tiết kiệm chính.
- **Bộ nhớ GPU** tăng theo phần mô hình phải lan truyền ngược: chỉ train head dùng 0,25–0,60 GB, fine-tune toàn bộ dùng 1,7–2,2 GB. Cả 8 cấu hình đều vừa GPU T4 16 GB với batch 128.
- **Tốc độ dự đoán** gần như bằng nhau (khoảng 1.900 ảnh/s) ở mọi cấu hình, vì chế độ huấn luyện không đổi kiến trúc lúc dự đoán, và ở batch 128 bước nạp dữ liệu và phóng ảnh chiếm phần lớn thời gian. Hai cấu hình chỉ train head có cùng 23 giây mỗi epoch vì cùng lý do.

9. Phân tích lỗi

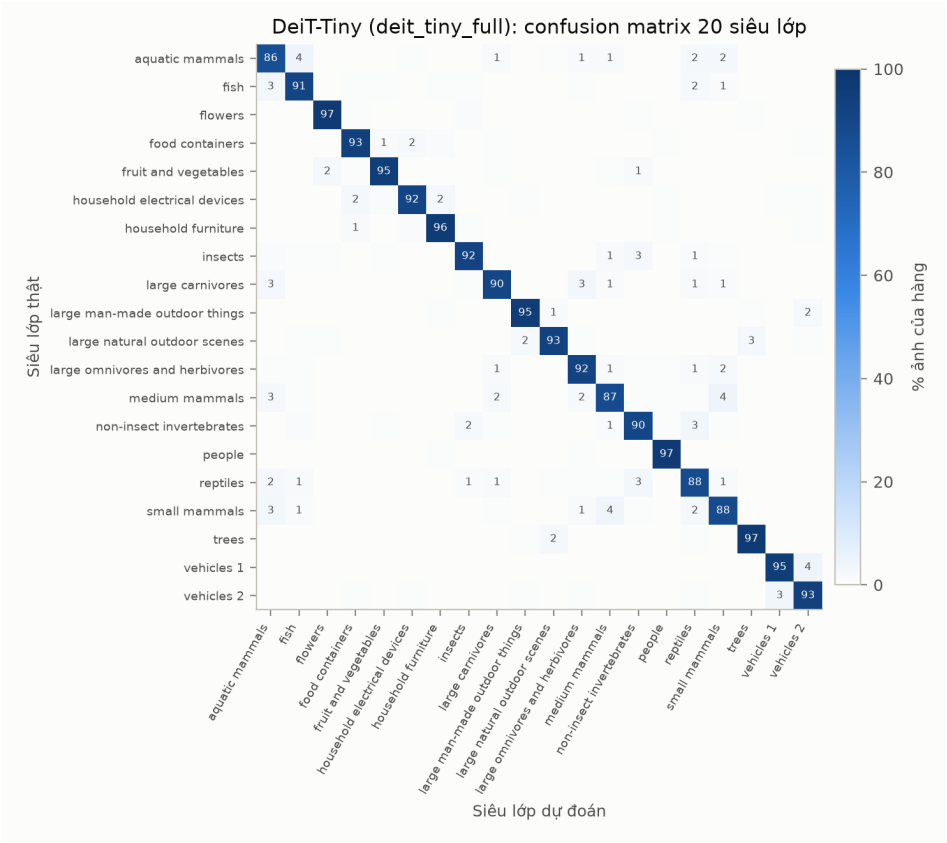
Phần này so sánh cấu hình tốt nhất của mỗi họ, chọn theo validation accuracy: `resnet18_full` (ResNet-18) và `deit_tiny_full` (DeiT-Tiny).

9.1. Nhầm lẫn theo siêu lớp

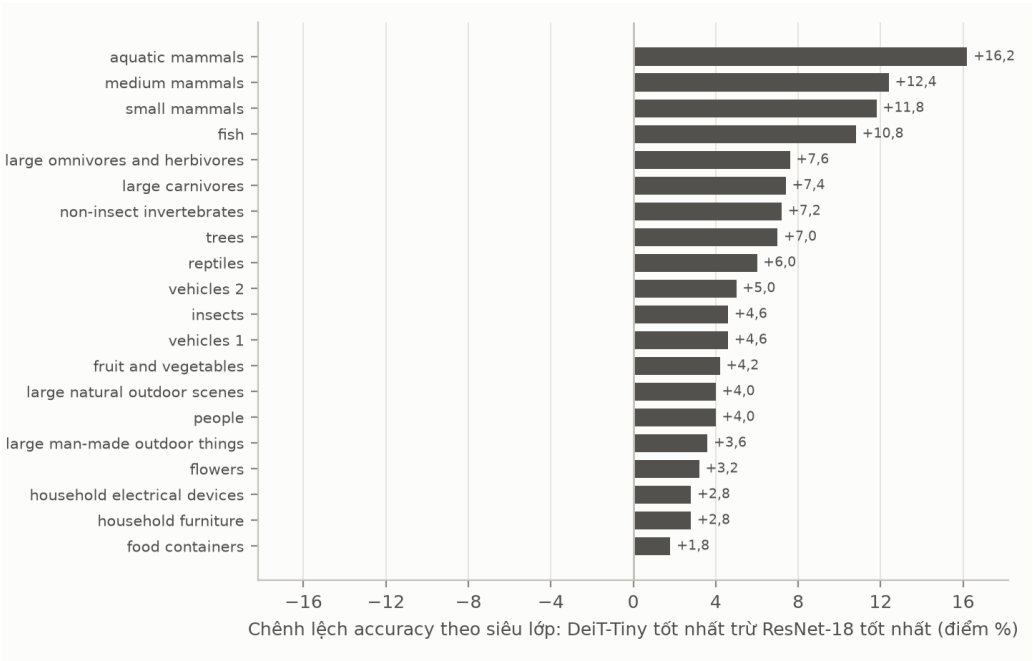
Confusion matrix 100×100 quá lớn để đọc, nên nhóm gom dự đoán về 20 siêu lớp (Bảng 2). Accuracy ở mức siêu lớp: ResNet-18 85,93%, DeiT-Tiny 92,28%. Trong số các ảnh bị đoán sai lớp chi tiết, tỉ lệ ảnh bị đoán nhầm sang một lớp *cùng siêu lớp* là 44,4% với ResNet-18 và 48,5% với DeiT-Tiny (nếu nhầm ngẫu nhiên, tỉ lệ này chỉ khoảng $4/99 \approx 4\%$).



Hình 7: Confusion matrix theo 20 siêu lớp của ResNet-18 tốt nhất (mỗi hàng chuẩn hoá về 100%).



Hình 8: Confusion matrix theo 20 siêu lớp của DeiT-Tiny tốt nhất.



Hình 9: Chênh lệch accuracy theo siêu lớp giữa DeiT-Tiny và ResNet-18 tốt nhất (dương: DeiT-Tiny tốt hơn).

9.2. Các cặp lớp hay nhầm nhất

Bảng 9: 8 cặp (lớp thật → lớp dự đoán) bị nhầm nhiều nhất trên 10.000 ảnh test (mỗi lớp có 100 ảnh).

ResNet-18 (`resnet18_full`)

Thật	Dự đoán	Số ảnh
maple tree	oak tree	29
girl	woman	27
pine tree	oak tree	24
willow tree	oak tree	21
seal	otter	21
boy	man	20
otter	seal	20
couch	bed	20

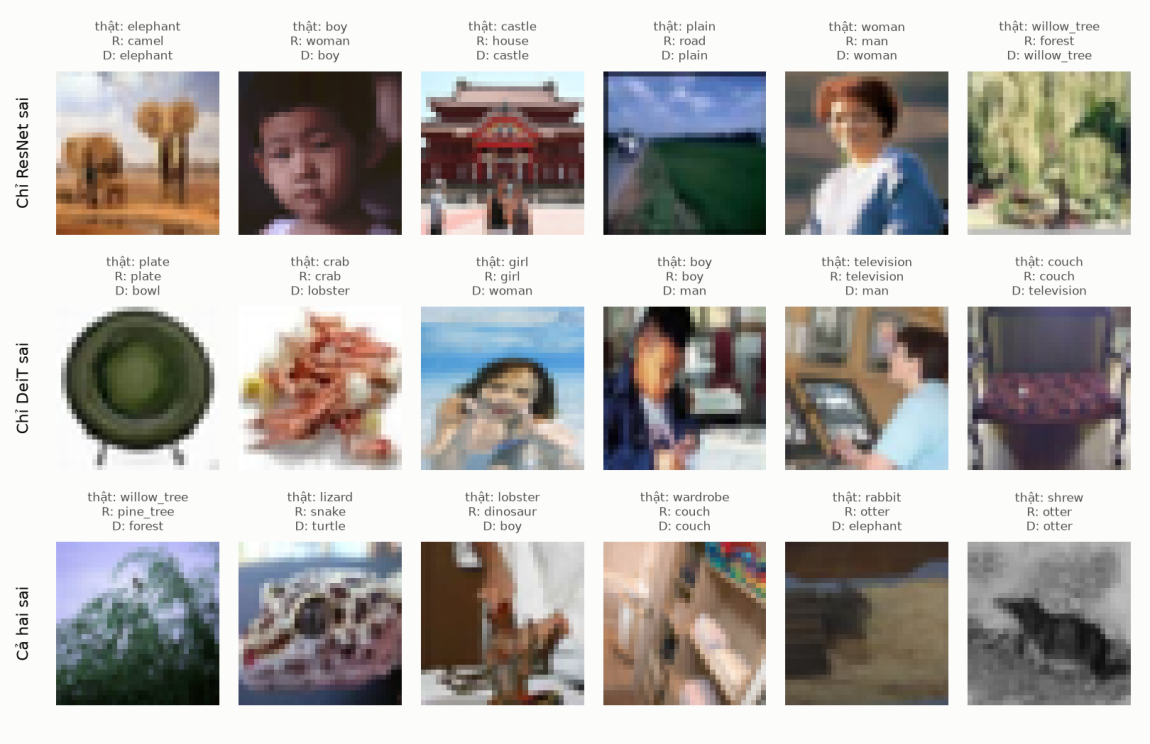
DeiT-Tiny (`deit_tiny_full`)

Thật	Dự đoán	Số ảnh
girl	woman	20
maple tree	oak tree	19
oak tree	maple tree	17
boy	man	16
man	boy	15
plate	bowl	13
woman	man	13
seal	otter	12

Năm lớp khó nhất (accuracy theo lớp): ResNet-18: girl (31%), otter (40%), seal (41%), beaver (41%), willow tree (44%); DeiT-Tiny: girl (58%), boy (58%), man (66%), maple tree (67%), woman (69%). Năm lớp dễ nhất: ResNet-18: motorcycle (96%), wardrobe (94%), bicycle (93%), orange (93%), television (93%); DeiT-Tiny: skunk (100%), road (99%), tractor (98%), motorcycle (97%), pickup truck (96%).

9.3. Hai mô hình có sai cùng chỗ không?

Trên 10.000 ảnh test: cả hai cùng đúng 70,59%, chỉ ResNet-18 đúng 4,12%, chỉ DeiT-Tiny đúng 14,43%, cả hai cùng sai 10,86%. Nếu luôn chọn được mô hình đúng trong hai, accuracy sẽ là 89,14% (giới hạn trên của việc kết hợp hai mô hình).



Hình 10: Ảnh test bị đoán sai, chọn ngẫu nhiên (seed 42) theo ba nhóm. "R" là dự đoán của ResNet-18, "D" là của DeiT-Tiny.

Nhận xét.

- **Lỗi tập trung trong cùng siêu lớp.** Gần một nửa số ảnh bị đoán sai bị nhầm sang lớp cùng siêu lớp, gấp hơn 10 lần mức ngẫu nhiên. Các cặp hay nhầm nhất ở cả hai mô hình gần như trùng nhau và đều là những lớp giống nhau về nghĩa: *girl / woman*, *boy / man*, *maple tree / oak tree*, *seal / otter*. Ở độ phân giải 32×32, phân biệt tuổi và giới tính của người hay loài cây chỉ dựa vào vài điểm ảnh, nên đây là lỗi khó tránh, không phải lỗi ngẫu nhiên.
- **Lớp khó.** Nhóm *people* (boy, girl, man, woman) nằm trong số lớp khó nhất của cả hai mô hình. Lớp dễ là các vật có hình dạng hoặc màu sắc đặc trưng (motorcycle, road, tractor, wardrobe).
- **DeiT-Tiny tốt hơn ở cả 20 siêu lớp** (Hình 9). Chênh lệch lớn nhất ở các nhóm động vật (aquatic mammals +16,2, medium mammals +12,4, small mammals +11,8, fish +10,8 điểm), nhỏ nhất ở đồ vật trong nhà (food containers +1,8, household furniture và electrical devices +2,8). Nhóm chưa kiểm chứng nguyên nhân; một giả thuyết là các lớp động vật cần phân biệt kết cấu lông, da và bối cảnh trên toàn ảnh, phù hợp với attention toàn cục, còn đồ vật có hình dạng rõ thì cả hai mô hình đều đã làm tốt.
- **Hai mô hình bổ sung cho nhau một phần.** Có 4,12% ảnh chỉ ResNet-18 đúng, nên nếu kết hợp lý tưởng hai mô hình thì accuracy có thể lên 89,14% (so với 85,02% của DeiT-Tiny). Đây chỉ là giới hạn trên; nhóm không thử ensemble trong bài này.
- **Ảnh cả hai cùng sai** (10,86%, hàng dưới của Hình 10) thường là ảnh mà vật chính bị che, chụp cận một phần, góc nhìn lạ hoặc lẫn vào nền (ví dụ con tôm hùm trong tay người, cận cảnh một góc tủ quần áo). Một số ảnh khó cả với người nhìn ở 32×32; cải thiện nhóm này cần dữ liệu độ phân giải cao hơn hơn là đổi kiến trúc.

10. Thảo luận

10.1. ViT có cần pretrain không?

Giả thuyết 1 được xác nhận. Học từ đầu với cùng 30 epoch và cùng augmentation, DeiT-Tiny chỉ đạt 48,48%, kém ResNet-18 (67,46%) 19,0 điểm, và overfit nặng hơn (Hình 5). Pretrain thay đổi hoàn toàn thứ tự: DeiT-Tiny tăng 36,5 điểm khi dùng trọng số pretrained (từ 48,48% lên 85,02%), trong khi ResNet-18 chỉ tăng 7,3 điểm (từ 67,46% lên 74,71%). Thậm chí DeiT-Tiny chỉ train head (65,11%, 19.300 tham số) vẫn tốt hơn DeiT-Tiny học từ đầu 5,5 triệu tham số. Kết quả này khớp với lập luận: tích chập có sẵn giả định "điểm ảnh gần nhau liên quan với nhau", nên học được từ 45.000 ảnh; Transformer phải tự học quan hệ không gian từ dữ liệu, và CIFAR-100 không đủ lớn để làm việc đó trong 30 epoch với augmentation nhẹ. Như đã ghi ở phần Hạn chế, đây là kết luận cho thiết lập của bài, không phải giới hạn tuyệt đối của kiến trúc.

10.2. Khi nào đóng băng có lợi?

Giả thuyết 2 đúng với ResNet-18, chỉ đúng một phần với DeiT-Tiny.

- **ResNet-18:** đóng băng một phần (73,67%) chỉ kém fine-tune toàn bộ (74,71%) 1,0 điểm, trong khi thời gian train giảm một nửa (6,0 so với 12,3 phút) và bộ nhớ giảm từ 1,69 xuống 0,72 GB. Đây là lựa chọn hợp lý khi tài nguyên hạn chế. Tuy nhiên tiết kiệm đến từ việc bỏ lan truyền ngược qua các tầng đầu, không phải từ số tham số (vẫn train 93,9%).
- **DeiT-Tiny:** đóng băng 6 khối đầu (81,31%) kém fine-tune toàn bộ (85,02%) 3,7 điểm, đổi lại train nhanh hơn khoảng 35% và tốn khoảng một nửa bộ nhớ, với 48,5% tham số trainable. Các khối đầu của DeiT-Tiny dường như vẫn cần điều chỉnh cho ảnh CIFAR phóng to, khác với các tầng đầu của CNN vốn học đặc trưng cấp thấp (cạnh, màu) dùng chung được.
- **Chỉ train head kém xa ở cả hai họ** (thấp hơn fine-tune toàn bộ 23,7 điểm với ResNet-18 và 19,9 điểm với DeiT-Tiny), đúng như dự đoán: đặc trưng ImageNet cố định không khớp đủ với ảnh 32×32 phóng lên. Chế độ này chỉ đáng dùng khi cần huấn luyện cực nhanh hoặc rất ít bộ nhớ.

10.3. Chi phí so với chất lượng

Giả thuyết 3 (so sánh accuracy trên mỗi đơn vị thời gian): trên GPU T4, fine-tune từ trọng số pretrained tốt hơn học từ đầu ở cả hai trục: `deit_tiny_full` đạt accuracy cao nhất (85,02%) với khoảng một phần ba thời gian của các cấu hình học từ đầu. Thứ tự đề xuất theo ngân sách:

- Khoảng 4 phút: `deit_tiny_head` (65,11%), không nên dùng `resnet18_head` (51,02%) với cùng chi phí.
- Khoảng 6–7 phút: `deit_tiny_partial` (81,31%) hoặc `resnet18_partial` (73,67%).
- Khoảng 11 phút: `deit_tiny_full` (85,02%), cấu hình tốt nhất của bài.

Học từ đầu chỉ có ý nghĩa khi không được dùng trọng số pretrained; khi đó nên chọn CNN (ResNet-18) thay vì ViT.

10.4. Hạn chế

- **Một seed cho mỗi cấu hình.** Do giới hạn GPU, mỗi cấu hình chỉ chạy một lần; chênh lệch nhỏ hơn khoảng 1 điểm phần trăm giữa hai cấu hình nên được xem là chưa chắc chắn.
- **Siêu tham số không được tinh chỉnh riêng.** Mỗi nhóm cấu hình dùng chung một

bộ siêu tham số (Bảng 6); mỗi cấu hình có thể đạt cao hơn nếu được tinh chỉnh riêng.
- **Augmentation nhẹ, ít epoch cho from-scratch.** Kết quả from-scratch (đặc biệt của DeiT-Tiny) phản ánh đúng thiết lập này, không phải giới hạn của kiến trúc: DeiT gốc học từ đầu trên ImageNet với augmentation mạnh và 300 epoch [4].
- **Ảnh 32×32 phóng lên 224×224** không thêm thông tin mới; chi phí tính toán tăng khoảng 49 lần so với chạy ở 32×32. Nhóm chấp nhận để dùng được trọng số pretrained đúng độ phân giải gốc.

11. Tái lập kết quả

- **Trên Google Colab:** mở `Assignment_1/colab_a1_2.ipynb`, chọn GPU T4 và *Run all*. Log được ghi vào Google Drive; nếu phiên bị ngắt, chỉ cần *Run all* lại.
- **Trên máy có GPU:** `pip install -r requirements.txt`, sau đó `python run_all.py` (8 thí nghiệm, log ở `output/runs/`) và `python analyze.py` (bảng, hình và số liệu của báo cáo này).
- Split cố định ở `splits/cifar100_split.json`, seed 42 cho mọi thí nghiệm. Log JSON của từng cấu hình (lịch sử theo epoch, chi phí, kết quả test) và dự đoán trên test được commit trong `output/runs/`; trọng số (`*.pth`) không commit vì dung lượng lớn.

12. Sử dụng công cụ AI

Theo quy định liêm chính học thuật của môn học, nhóm ghi rõ: trợ lý AI (Claude) được dùng để hỗ trợ viết khung mã nguồn (huấn luyện, phân tích, vẽ biểu đồ), soạn bố cục báo cáo và rà soát lỗi. Mọi số liệu trong báo cáo được sinh tự động từ log huấn luyện thật bằng `analyze.py`, không nhập tay. Nhóm đã đọc, chạy kiểm tra, chỉnh sửa và chịu trách nhiệm về toàn bộ nội dung; mọi lựa chọn dataset, mô hình, protocol và các nhận xét đều được nhóm hiểu và có thể giải thích.

Tài liệu

[1] A. Krizhevsky. *Learning Multiple Layers of Features from Tiny Images*. Technical report, University of Toronto, 2009.

[2] K. He, X. Zhang, S. Ren, J. Sun. Deep Residual Learning for Image Recognition. *CVPR*, 2016.

[3] A. Dosovitskiy et al. An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. *ICLR*, 2021.

[4] H. Touvron et al. Training data-efficient image transformers & distillation through attention. *ICML*, 2021.

[5] R. Wightman. PyTorch Image Models (timm). `https://github.com/huggingface/pytorch-image-models`.

[6] P. Micikevicius et al. Mixed Precision Training. *ICLR*, 2018.